Exbooks 系統架構

accounts, books, deals 模組設計

資料夾結構與職責劃分

整體架構概覽

專案結構設計哲學

```
exbook/
├── accounts/      # 用戶與身份認證
├── books/         # 書籍核心管理
├── deals/         # 交易與借閱系統
├── core/          # 抽象基類與工具
└── config/        # 專案配置
```

accounts/ - 用戶身份認證

職責範圍

- 用戶註冊與身份驗證
- 用戶個人資料管理
- 權限與安全控制
- 用戶信用評價系統

核心檔案結構

```
accounts/
├── models.py              # UserProfile, Appeal, Rating
├── views.py               # 個人頁面、評價查詢
├── forms.py               # 註冊表單、個人資料表單
├── adapters.py            # django-allauth 適配器
├── validators.py          # 年齡驗證、資料驗證
├── services/
│   ├── trust_service.py   # 信用積分計算
│   └── appeal_service.py  # 申訴處理
└── tests/                 # 單元測試
```

關鍵設計決策

- **移除 username**：使用 email 作為唯一識別
- **年齡驗證**：BR-11 規則（年滿 18 歲）
- **信用系統**：不獨立建表，動態計算
- **整合 allauth**：支援 Google OAuth + Email 登入

books/ - 書籍核心管理

職責範圍

- 書籍目錄與元資料管理
- 書況照片處理與儲存
- 書籍搜尋與篩選功能
- 願望書單與通知系統

核心檔案結構

```
books/
├── models/
│   ├── __init__.py
│   ├── official_book.py    # 官方書籍資訊
│   ├── shared_book.py     # 共享書籍實例
│   ├── book_set.py        # 套書綁定
│   └── book_photo.py      # 書況照片
├── services/
│   ├── isbn_service.py     # ISBN 查詢
│   ├── photo_service.py   # 照片處理
│   └── wishlist_service.py
├── views.py                # 書籍CRUD、搜尋
├── forms.py                # 書籍上架表單
└── tests/                  # 測試檔案
```

deals/ - 交易與借閱系統

職責範圍

- 借閱交易管理
- 交易訊息協調
- 借閱期限與延期
- 通知與提醒系統

核心檔案結構

```
deals/
├── models/
│   ├── __init__.py
│   ├── deal.py            # 交易主體
│   ├── deal_message.py    # 交易訊息
│   ├── rating.py          # 評價系統
│   ├── loan_extension.py  # 延期申請
│   ├── notification.py     # 通知訊息
│   └── push_subscription.py
├── services/
│   ├── deal_service.py     # 交易核心邏輯
│   ├── extension_service.py
│   ├── rating_service.py
│   ├── notification_service.py
│   └── push_service.py
├── views.py                # 交易相關視圖
└── tests/                  # 測試檔案
```

業務邏輯封裝

core/ - 抽象基類與工具

職責範圍

- 提供抽象基類
- 共用工具函數
- 通用功能封裝

核心設計

```python
# core/models.py
class BaseModel(models.Model):
    """所有模型的基類"""
    id = models.UUIDField(primary_key=True, default=uuid.uuid4)
    created_at = models.DateTimeField(auto_now_add=True)

    class Meta:
        abstract = True

class UpdatableModel(BaseModel):
    """可更新的模型基類"""
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        abstract = True
```

模組間的協同工作

資料流設計

```
graph TD
    A[accounts] -->|用戶資料| B[books]
    B -->|書籍狀態| C[deals]
    C -->|交易結果| A

    subgraph 核心流程
        A1[用戶註冊] --> B1[書籍上架]
        B1 --> C1[發起交易]
        C1 --> D1[面交完成]
        D1 --> E1[相互評價]
    end
```

API 設計原則

- **RESTful 設計**：清晰資源導向
- **權限控制**：基於狀態與角色的權限
- **版本管理**：為未來 API 版本預留
- **錯誤處理**：一致的錯誤回應格式

測試架構設計

測試金字塔實踐

單元測試

- Model 極證
- Service 邏輯
- 業務規則測試
- **196+ 測試案例**

整合測試

- API 端點測試
- 模組整合測試
- 資料庫操作測試

E2E 測試

- 使用者流程測試
- 瀏覽器自動化
- 真實場景驗證
- **308+ 測試案例**

技術決策總結

為什麼這樣設計？

領域驅動

- 按業務領域劃分模組
- 每個模組職責單一
- 利於團隊協作開發

模組化設計

- 高內聚低耦合
- 易於測試與維護
- 可獨立部署擴展

服務層抽象

- 複雜邏輯集中管理
- 業務規則統一執行
- 利於未來重構